Проект 22 — Agent Algorithms: UML Data Flow

Источники: coding-agent-algorithms.jsx · Aider · LangChain · LangGraph. Ноль внешних пакетов.

Диаграмма 1 — 8 групп алгоритмов: структура и зависимости

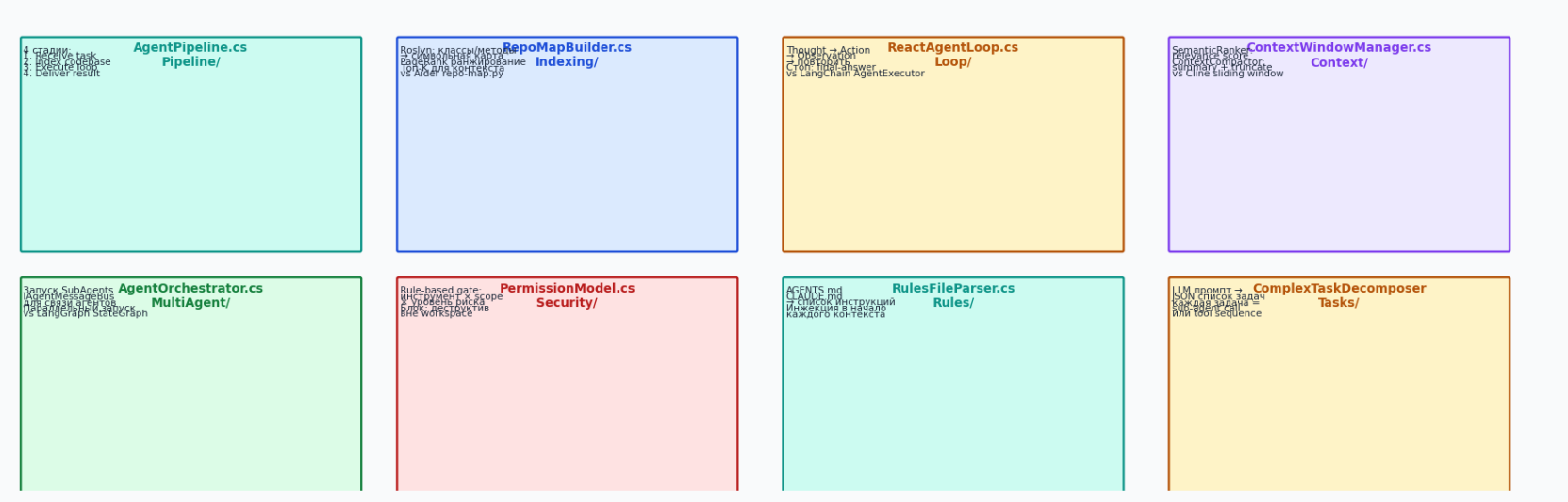


Диаграмма 2 — ReAct цикл (ReactAgentLoop.cs) + ContextWindowManager

Источник: ReAct — Yao et al. 2022. Аналог: LangChain AgentExecutor, OpenCode ExecutionEngine.

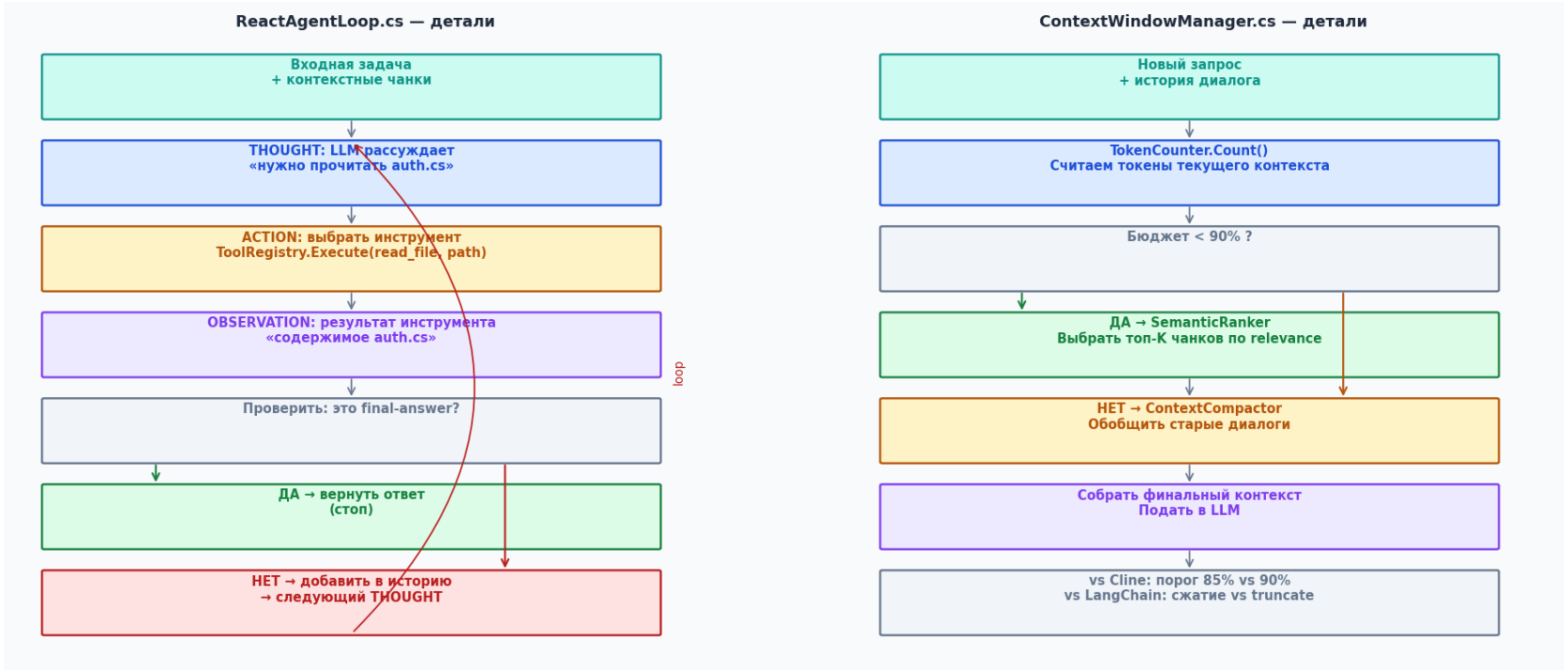

ReactAgentLoop.cs — детали
Входная задача
+ контекстные чанки
THOUGHT: LLM рассуждает
«нужно прочитать auth.cs»
ACTION: выбрать инструмент
ToolRegistry.Execute(read_file, path)
OBSERVATION: результат инструмента
«содержимое auth.cs»
Проверить: это final-answer?
ДА → вернуть ответ
(стоп)
НЕТ → добавить в историю
→ следующий THOUGHT
loop
ContextWindowManager.cs — детали
Новый запрос
+ история диалога
TokenCounter.Count()
Считаем токены текущего контекста
Бюджет < 90% ?
ДА → SemanticRanker
Выбрать топ-K чанков по relevance
НЕТ → ContextCompactor
Обобщить старые диалоги
Собрать финальный контекст
Подать в LLM
vs Cline: порог 85% vs 90%
vs LangChain: сжатие vs truncate

Диаграмма 3 — AgentOrchestrator: многоагентная оркестрация

vs LangGraph StateGraph: схожая концепция узлов и переходов. Наш вариант — C# async/await без граф-DSL.

